

TestPlatform

Rapport d'Execution de Campagne

Campagne #19

Projet: Projet #17

Version: N/A

Environnement: Environnement #12

Date: 03/06/2026 10:46

REUSSI

Resume executif

Taux de reussite



Total	Succes	Echecs	Erreurs	Duree
1	1	0	0	313.3 s

Informations du contexte

Projet	Projet #17
Description	N/A

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
Test #32	UNIT	N/A	N/A	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Test #32

Type	UNIT
Script	IA

SUCCESS	
Duree	11.4 s
Erreur	N/A

Analyse IA

Test #32

Logs (extrait)	<pre>----- ----- STDERR ----- Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended</pre>
Analyse IA	Analyse du script de test : 1. Ce qui a bien fonctionné : Le script de test semble être structuré correctement et les mocks sont configurés correctement pour chaque repository et service nécessaires à l'exécution des tests unitaires de `TestCaseService`. La configuration du contexte est réalisée avec `MockitoAnnotations.openMocks(this)`, ce qui permet d'initialiser les mocks avant chaque test. 2. Suggestion d'optimisation : Pour l'optimisation des temps d'exécution ou de la robustesse du script, voici quelques suggestions : - La configuration des réponses aux appels à `SecurityUtils` et au repository peut être centralisée dans un bloc `@BeforeAll` pour éviter une duplication de code. - Les mocks sont créés pour chaque test, ce qui peut être optimisé en utilisant la configuration statique (static mocking) fournie par Mockito 4+. Cela permet d'avoir un seul mock perpétuellement en place dans tous les tests du package. - Le code source est compilé à chaque fois qu'une modification de code est effectuée, ce qui peut être amélioré en mettant cela hors de l'éditeur (CI/CD) et déploiement uniquement les classes compilées. - Envisagez d'utiliser des annotations TestNG comme `@BeforeClass` pour ouvrir le contexte, etc., au lieu de mocks dans chaque test. Cela peut également améliorer les temps d'exécution en éliminant la duplication de code.

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
Test #32	Suite UNIT	11.4 s

Temps d'execution par type

Type	Duree moyenne
UNIT	11.4 s

Tests instables

Aucun test instable detecte

Recommandations

Aucune recommandation spécifique

Annexe - Scripts generes par IA

Test #32

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```
suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
    Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));

Mockito.when(llmService.generateTestCode(
    Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");

Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
        TestCase tc = inv.getArgument(0);
        tc.setSuite(suite);
        return tc;
    });

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
// verify
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any());
}
```